

算法分析第3次作业

算法实现题4

4-1

4-1 会场安排问题。

问题描述：假设要在足够多的会场里安排一批活动，并希望使用尽可能少的会场。设计一个有效的贪心算法进行安排。（这个问题实际上是著名的图着色问题。若将每个活动作为图的一个顶点，不相容活动间用边相连。使相邻顶点着有不同颜色的最小着色数，相当于要找的最小会场数。）

算法设计：对于给定的 k 个待安排的活动，计算使用最少会场的时间表。

数据输入：由文件 `input.txt` 给出输入数据。第 1 行有 1 个正整数 k ，表示有 k 个待安排的活动。接下来的 k 行中，每行有 2 个正整数，分别表示 k 个待安排的活动开始时间和结束时间。时间以 0 点开始的分钟计。

结果输出：将计算的最少会场数输出到文件 `output.txt`。

113

输入文件示例

`input.txt`

5

1 23

12 28

25 35

27 80

36 50

输出文件示例

`output.txt`

3

第4-1题答案:

一、算法设计思想

本问题旨在求出满足所有活动互不重叠所需的最少会场数量。该问题可以转化为求解所有时刻中，相互重叠的活动数量的最大值。为了高效统计这一峰值，可以采用基于事件扫描（扫描线）的贪心算法，具体设计思想如下：

- 1. 端点拆分与事件定义：** 将给定的 k 个活动的开始时间和结束时间分离，转化为 $2k$ 个独立的“事件”或“端点”。每个端点包含两个属性：时间值以及端点类型（“开始”或“结束”）。
- 2. 事件排序：** 将这 $2k$ 个端点按照时间值进行升序排序。
 - **关键处理规则：** 当某个活动的结束时间与另一个活动的开始时间相同时，由于不相容活动的定义，前一个活动结束后释放的会场可以立即供后一个活动使用。因此，在排序时，对于时间值相同的端点，必须规定“结束”端点排在“开始”端点之前，以确保优先释放会场。
- 3. 贪心扫描与状态维护：** 初始化两个计数器变量：`curr` 用于记录当前正在使用的会场数量（初始为0），`sum` 用于记录整个过程中同时使用会场数量的最大值（初始为0）。
- 4. 线性遍历：** 顺序遍历排序后的 $2k$ 个端点：
 - 若当前端点为“开始”端点，表示一个新的活动开始，需要占用一个会场，执行 `curr++`。
 - 若当前端点为“结束”端点，表示一个活动结束，释放一个会场，执行 `curr--`。
 - 在每次更新 `curr` 后，比较并更新最大需求量，即 `sum = max(sum, curr)`。
- 5. 结果输出：** 遍历结束后，`sum` 中保存的值即为安排所有活动所需的最少会场总数。

二、算法分析

1. 正确性分析：

该贪心策略的本质是寻找“最大重叠区间数”。将每个活动视为时间轴上的一个区间，问题实质上是求解区间图的最小着色数。根据图论性质，对于区间图而言，其最小着色数等于图的最大团大小（即同一时刻相互重叠的区间的最大数量）。

通过将时间点离散化为事件并按时间先后顺序处理，计数器 `curr` 实时且精确地记录了任意时刻系统中处于活跃状态（正在进行且尚未结束）的活动总数。由于在任何一个特定时刻，所有处于活跃状态的活动都必须被分配到不同的会场中，因此 `curr` 的历史最大值 `sum` 就是所需会场数量的理论下界。由于只要有空闲会场就可以直接分配（通过“结束”端点优先于“开始”端点的排序规则保证了复用），该下界是可以实际达到的最优解。因此，该贪心算法能够保证求得全局最优的最少会场数。

2. 时间复杂度分析：

- 将 k 个活动拆分为 $2k$ 个端点，需要进行常数次操作的循环，时间复杂度为 $O(k)$ 。
- 对 $2k$ 个端点进行排序，采用基于比较的高效排序算法（如快速排序或归并排序），时间复杂度为 $O(k \log k)$ 。
- 对排序后的端点数组进行一次线性扫描以统计最大会场数，需要遍历 $2k$ 次，时间复杂度为 $O(k)$ 。
- 综上所述，算法整体的时间复杂度主要由排序步骤决定，为 $O(k \log k)$ 。

3. 空间复杂度分析：

算法在执行过程中，除了输入数据外，需要额外构建一个长度为 $2k$ 的数据结构（如数组或列表）

来存储所有拆分后的端点信息。除此之外，仅需要维护常数个辅助变量（如 `curr`，`sum`）。因此，算法的空间复杂度为 $O(k)$ 。

一、算法设计思想

4-2

4-2 最优合并问题。

问题描述：给定 k 个排好序的序列 $s_1, s_2, \dots, s_k$ ，用 2 路合并算法将这 k 个序列合并成一个序列。假设采用的 2 路合并算法合并 2 个长度分别为 m 和 n 的序列需要 $m+n-1$ 次比较。试设计一个算法确定合并这个序列的最优合并顺序，使所需的总比较次数最少。

为了进行比较，还需要确定合并这个序列的最差合并顺序，使所需的总比较次数最多。

算法设计：对于给定的 k 个待合并序列，计算最多比较次数和最少比较次数合并方案。

数据输入：由文件 `input.txt` 给出输入数据。第 1 行有 1 个正整数 k ，表示有 k 个待合并序列。接下来的 1 行中，有 k 个正整数，表示 k 个待合并序列的长度。

结果输出：将计算的最多比较次数和最少比较次数输出到文件 `output.txt`。

输入文件示例

`input.txt`

4

5 12 11 2

输出文件示例

`output.txt`

78 52

第4-2题答案:

一、算法设计思想

本问题要求在给定的 k 个有序序列中分别找到合并所需的总比较次数最少（最优）方案和最多（最差）方案。已知每次进行 2 路合并时，长度为 m 和 n 的两个序列需要进行 $m+n-1$ 次比较。经过 $k-1$ 次合并后，所有序列合并为一个。设第 i 次合并的两个序列长度分别为 x_i 和 y_i ，则合并过程的总比较次数为 $\sum_{i=1}^{k-1} (x_i + y_i - 1) = \sum_{i=1}^{k-1} (x_i + y_i) - (k-1)$ 。由于常数项 $k-1$ 固定不变，极值化总比较次数等价于极值化合并产生的新序列长度之和 $\sum_{i=1}^{k-1} (x_i + y_i)$ 。该过程可完全映射为构建哈夫曼树（Huffman Tree）的思想：

• 最少比较次数（最优合并）：

为了使总比较次数最少，应采用贪心策略，每次优先选择当前长度**最短**的两个序列进行合并。具体实现时，可利用一个最小堆（优先队列）维护所有待合并的序列长度。每次从最小堆中弹出两个最小值 a 和 b ，将其累加到总比较次数中（增加 $a+b-1$ ），然后将新合并的长度 $a+b$ 重新压入最小堆。重复此过程 $k-1$ 次，直到堆中仅剩一个元素，即可得到最少比较次数。

• 最多比较次数（最差合并）：

为了使总比较次数最多，应采用相反的贪心策略，每次优先选择当前长度**最长**的两个序列进行合并。具体实现时，可利用一个最大堆维护所有序列长度，每次弹出两个最大值进行合并，并将新长度压回最大堆中；或者由于每次合并产生的两个最大序列之和必定大于其余所有未合并序列，直接

将原始数据按降序排序后，从头开始依次累积合并即可。重复此过程 $k - 1$ 次，即可得到最多比较次数。

二、算法分析

- **正确性分析：**

最优合并策略与哈夫曼树算法完全一致。每次合并相当于在哈夫曼树中生成一个内部节点，总比较次数等价于树中所有内部节点权值之和减去 $k - 1$ 。每次选取最小的两个元素合并，保证了数值越大的序列参与合并的次数越少（在树中的深度越浅），从而使得全局总合并代价最小。反之，最差合并策略每次将最大的两个元素优先合并，使得大数尽可能早地参与累加，并在后续合并中被重复累加的次数最多（在树中的深度最深），从而确保获得最大的总代价。此贪心选择性质保证了算法的正确性。

- **时间复杂度分析：**

对于最少比较次数的计算，构建包含 k 个元素的最小堆需要的时间为 $O(k)$ 。随后需要进行 $k - 1$ 次合并操作，每次操作包含出堆和入堆调整，单次堆调整的时间复杂度为 $O(\log k)$ ，故该部分时间复杂度为 $O(k \log k)$ 。对于最多比较次数的计算，使用最大堆的时间复杂度同理为 $O(k \log k)$ （若采用降序排序后顺序合并的方法，排序阶段同样需要 $O(k \log k)$ 时间，后续顺序合并为 $O(k)$ ）。综合来看，整个算法的时间复杂度为 $O(k \log k)$ 。

- **空间复杂度分析：**

算法在执行过程中，需要开辟额外的数据结构（最小堆、最大堆或数组）来存储和动态调整这 k 个序列的长度数值。其占用的最大内存空间与序列的初始个数 k 呈正比关系。因此，算法的空间复杂度为 $O(k)$ 。

4-4 磁盘文件最优存储问题。

问题描述：设磁盘上有 n 个文件 $f_1, f_2, \dots, f_n$ ，每个文件占用磁盘上的 1 个磁道。这 n 个文件的检索概率分别是 $p_1, p_2, \dots, p_n$ 且 $\sum_{i=1}^n p_i = 1$ 。磁头从当前磁道移到被检信息磁道所需的时间可用这两个磁道之间的径向距离来度量。如果文件 f_i 存放在第 i ($1 \leq i \leq n$) 道上，则检索这 n 个文件的期望时间是 $\sum_{1 \leq i < j \leq n} p_i p_j d(i, j)$ 。式中， $d(i, j)$ 是第 i 道与第 j 道之间的径向距离 $|i - j|$ 。

磁盘文件的最优存储问题要求确定这 n 个文件在磁盘上的存储位置，使期望检索时间达到最小。试设计一个解此问题的算法，并分析算法的正确性与计算复杂性。

算法设计：对于给定的文件检索概率，计算磁盘文件的最优存储方案。

数据输入：由文件 input.txt 给出输入数据。第 1 行是正整数 n ，表示文件个数。第 2 行有 n 个正整数 a_i ，表示文件的检索概率。实际上第 k 个文件的检索概率应为 $a_k / \sum_{i=1}^n a_i$ 。

结果输出：将计算的最小期望检索时间输出到文件 output.txt。

输入文件示例	输出文件示例
input.txt	output.txt
5	0.547396
33 55 22 11 9	

第4-4题答案:

一、 算法设计思想

磁盘文件最优存储问题的核心在于通过合理安排 n 个文件在磁道上的物理顺序，使得加权期望检索时间 $\sum_{1 \leq i < j \leq n} p_i p_j |i - j|$ 达到最小。由于检索概率 p_i 越大的文件对总期望值的影响权重越高，根据重排序不等式的直观推导，应将概率较大的文件尽可能靠近放置，以减小其相互之间的径向距离 $|i - j|$ 。

具体的贪心算法设计步骤如下：

1. 概率归一化与排序：首先，根据输入数据计算每个文件的实际检索概率 $p_i = a_i / \sum_{j=1}^n a_j$ 。接着，将所有文件按概率从大到小进行排序，得到序列 $f_{(1)}, f_{(2)}, \dots, f_{(n)}$ ，满足 $p_{(1)} \geq p_{(2)} \geq \dots \geq p_{(n)}$ 。

2. 磁道分配策略（中心扩散法）：

为了使概率最高的文件与其他所有文件的距离总和最小，应采用从中心向两翼交替放置的策略：

- 将概率最大的文件 $f_{(1)}$ 放置在 n 个磁道的中间位置（若磁道编号为 0 至 $n - 1$ ，则位置为 $k = \lfloor (n - 1) / 2 \rfloor$ ）。
- 将后续概率次大的文件依次放置在已分配区域的左右两侧。具体规则为： $f_{(2)}$ 放置在 $f_{(1)}$ 的左侧， $f_{(3)}$ 放置在 $f_{(1)}$ 的右侧， $f_{(4)}$ 放置在 $f_{(2)}$ 的左侧， $f_{(5)}$ 放置在 $f_{(3)}$ 的右侧，以此类推。

- 最终形成的布局在概率分布上呈现“中间高、两边低”的形态（类似正态分布或山峰状），确保了高概率文件之间的位移距离最小化。

3. 计算最小期望值：

根据最终确定的文件存储位置 $x[0], x[1], \dots, x[n-1]$ （其中 $x[i]$ 为第 i 条磁道上文件的概率值），代入公式 $E = \sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} x[i] \cdot x[j] \cdot (j-i)$ 计算最终的最小期望检索时间。

二、算法分析

1. 正确性分析：

本问题的目标函数是关于位置距离的加权求和。根据重排序原理，要使乘积之和最小，应当使较大的系数与较小的距离相乘。将概率最大的文件置于中心，能使其与所有其他位置的平均距离最短。后续文件按概率降序向两边排布，保证了任意两个高概率文件之间的距离都保持在较低水平。这种对称式的贪心策略在处理此类线性排列的距离加权最小化问题时，能有效达到全局最优解。

2. 计算复杂度分析：

- **时间复杂度：** * 对 n 个文件按概率进行排序需要 $O(n \log n)$ 。
 - 通过线性遍历进行中心扩散排列需要 $O(n)$ 。
 - 计算最终的期望值时，需要进行双重循环遍历，时间复杂度为 $O(n^2)$ 。因此，整个算法的时间复杂度主要取决于结果计算部分，为 $O(n^2)$ 。若仅考虑存储方案的设计，则复杂度为 $O(n \log n)$ 。
- **空间复杂度：** 算法需要辅助数组存储排序后的概率以及最终的磁道分布序列，其规模与文件数量 n 成正比。因此，空间复杂度为 $O(n)$ 。

4-6 最优服务次序问题。

问题描述：设有 n 个顾客同时等待一项服务，顾客 i 需要的服务时间为 t_i ($1 \leq i \leq n$)。应如何安排 n 个顾客的服务次序才能使平均等待时间达到最小？平均等待时间是 n 个顾客等待服务时间的总和除以 n 。

算法设计：对于给定的 n 个顾客需要的服务时间，计算最优服务次序。

数据输入：由文件 input.txt 给出输入数据。第 1 行是正整数 n ，表示有 n 个顾客。接下来的 1 行中，有 n 个正整数，表示 n 个顾客需要的服务时间。

结果输出：将计算的最小平均等待时间输出到文件 output.txt。

输入文件示例

input.txt

10

56 12 1 99 1000 234 33 55 99 812

输出文件示例

output.txt

532.00

第4-6题答案:

一、 算法设计思想

最优服务次序问题要求确定 n 个顾客的服务顺序，以使平均等待时间达到最小。平均等待时间最小化等价于总等待时间最小化。

1. 等待时间的形式化表达：

设 n 个顾客需要的服务时间分别为 $\{t_1, t_2, \dots, t_n\}$ 。假设服务顺序为 $1, 2, \dots, n$ ，对应的服务时间序列为 $t_{(1)}, t_{(2)}, \dots, t_{(n)}$ 。

- 第 1 个顾客的等待时间为：0
- 第 2 个顾客的等待时间为： $t_{(1)}$
- 第 3 个顾客的等待时间为： $t_{(1)} + t_{(2)}$
- 第 i 个顾客的等待时间为： $\sum_{j=1}^{i-1} t_{(j)}$

2. 总等待时间计算：

所有顾客的总等待时间 T 为：

$$T = \sum_{i=1}^n \sum_{j=1}^{i-1} t_{(j)} = (n-1)t_{(1)} + (n-2)t_{(2)} + \dots + 1 \cdot t_{(n-1)} + 0 \cdot t_{(n)}$$

即 $T = \sum_{i=1}^n (n-i)t_{(i)}$ 。

3. 贪心策略选择：

根据上述公式，服务时间 $t_{(i)}$ 的权重为 $(n-i)$ 。为了使乘积之和 T 最小，根据**排序不等式**（反序和最小），应当将较小的服务时间赋予较大的权重。由于权重序列满足 $(n-1) > (n-2) > \dots > 0$ ，因此最优的服务时间序列应当满足非减次序，即 $t_{(1)} \leq t_{(2)} \leq \dots \leq t_{(n)}$ 。

贪心策略定为：短服务时间优先（Shortest Service Time First）。

4. 算法步骤：

- 将输入的 n 个顾客的服务时间按升序（从小到大）进行排序。
- 按照排序后的顺序计算每个顾客的等待时间。
- 累加所有顾客的等待时间并除以 n ，得到最小平均等待时间。

二、算法分析

1. 正确性分析：

采用交换论证法（Exchange Argument）证明贪心选择性质。假设在某种最优服务次序中，存在两个相邻的顾客 i 和 $i+1$ ，其服务时间满足 $t_{(i)} > t_{(i+1)}$ 。若交换这两者的服务顺序，其他所有顾客的等待时间均不会改变（排在前面的顾客不受影响，排在后面的顾客由于前序总服务时间之和不变也不受影响）。交换前，这两人的等待时间贡献为 $W_i + (W_i + t_{(i)})$ ；交换后，贡献变为 $W_i + (W_i + t_{(i+1)})$ 。因为 $t_{(i)} > t_{(i+1)}$ ，交换后的总等待时间必然减小，这与原次序是最优的假设矛盾。因此，最优次序中服务时间必须是非减的。

2. 时间复杂度分析：

- **排序阶段：**使用高效的排序算法（如快速排序或归并排序）对 n 个服务时间进行排序，时间复杂度为 $O(n \log n)$ 。
- **统计阶段：**线性遍历排序后的序列计算总等待时间，时间复杂度为 $O(n)$ 。
- **总复杂度：**算法的总时间复杂度为 $O(n \log n)$ 。

3. 空间复杂度分析：

算法需要存储 n 个顾客的服务时间数据，其空间占用与输入规模成正比。因此，空间复杂度为 $O(n)$ 。

4-8 d 森林问题。

问题描述： 设 T 是一棵带权树，树的每条边带一个正权， S 是 T 的顶点集， T/S 是从树 T 中将 S 中顶点删去后得到的森林。如果 T/S 中所有树的从根到叶的路长都不超过 d ，则称 T/S 是一个 d 森林。

- ① 设计一个算法求 T 的最小顶点集 S ，使 T/S 是 d 森林（提示：从叶向根移动）。
- ② 分析算法的正确性和计算复杂性。
- ③ 设 T 中有 n 个顶点，则算法的计算时间复杂性应为 $O(n)$ 。

算法设计： 对于给定的带权树，计算最小分离集 S 。

数据输入： 由文件 input.txt 给出输入数据。第 1 行有 1 个正整数 n ，表示给定的带权树有 n 个顶点，编号为 $1, 2, \dots, n$ 。编号为 1 的顶点是树根。接下来的 n 行中，第 $i+1$ 行描述与 i 个顶点相关联的边的信息。每行的第 1 个正整数 k 表示与该顶点相关联的边数。其后 $2k$ 个数中，每 2 个数表示 1 条边。第 1 个数是与该顶点相关联的另一个顶点的编号，第 2 个数是边权值。 $k=0$ ，表示相应的结点是叶结点。文件的最后一行是正整数 d ，表示森林中所有树的从根到叶的路长都不超过 d 。

结果输出： 将计算的最小分离集 S 的顶点数输出到文件 output.txt。如果无法得到所要求的 d 森林则输出 “No Solution!”。

输入文件示例	输出文件示例
input.txt	output.txt
4	1
2 2 3 3 1	
1 4 2	
0	
0	
4	

第4-8题答案:

一、 算法设计思想

本问题要求从带权树 T 中寻找最小顶点集 S ，使得删除 S 后生成的 d -森林中，所有树从根到叶的路径长度均不超过 d 。为了保证 S 的规模最小，应采用**自底向上（从叶向根）**的贪心策略。其核心思想是：尽可能推迟顶点的删除，只有当某条从当前节点向下的路径长度即将超过 d 时，才通过删除其父节点来阻断该路径向上传播。

具体算法设计如下：

- 1. **状态定义：** 对树中的每个顶点 u ，定义 $f(u)$ 为在当前森林中，以 u 为根的子树中，从 u 到其所有后代叶结点的**最长路径长度**。
- 2. **初始化：** * 若 u 是叶结点，则 $f(u) = 0$ 。
 - 若树中存在任何一条边的边权 $w > d$ ，则无论如何删除顶点都无法满足 d -森林的定义，直接输

出 “No Solution!”。

3. 自底向上遍历（后序遍历）： 对于每一个非叶结点 u ，遍历其所有子结点 v （设边 (u, v) 的权值为 w_{uv} ）：

- **贪心判断：** 检查是否存在某个子结点 v 满足 $f(v) + w_{uv} > d$ 。
- **执行删除：** * 如果存在至少一个子结点满足上述条件，说明如果保留顶点 u ，则从 u （或其祖先）出发经过 v 到达叶结点的路径长度必将超过 d 。
 - 根据贪心策略，此时必须将 u 归入集合 S 中（即删除 u ）。
 - 这样做不仅阻断了当前所有过长的向下路径，且由于 u 被删除，其父节点在处理时将不再受到 u 子树路径长度的影响，从而最大程度地保护了祖先节点不被删除。
- **状态更新：** * 若 $u \in S$ ，则 u 对其父节点贡献的路径长度视为 0（或不再参与父节点的计算）。
 - 若 $u \notin S$ ，则 $f(u) = \max\{f(v) + w_{uv}\}$ ，其中 v 遍历 u 的所有子结点。

4. 统计结果： 遍历结束后，集合 S 的元素个数即为所求的最小分离集大小。

二、算法分析

1. 正确性分析（贪心证明）：

- **贪心选择性质：** 考虑任意一条长度超过 d 的路径。为了破坏这条路径，必须删除该路径上的至少一个顶点。在自底向上的过程中，当我们发现 u 节点与其子树形成的路径将要超过 d 时，选择删除 u 是最优的。因为删除 u 不仅能解决以 u 为“局部根”的所有超长路径问题，还能最有效地减小 u 的父节点向上传递的路径长度，从而降低父节点或更高层祖先被删除的可能性。
- **最优子结构：** 问题的最优解包含其子树在满足条件下的最优解。通过局部最优的删除决策，最终可以推导出全局顶点的最小化。

2. 计算复杂度分析：

- **时间复杂度：** 算法只需要对树进行一次完整的遍历（如后序遍历或利用度数为1的节点进行拓扑式扫描）。在处理每个顶点时，只需访问其关联的边。由于树中边数 $m = n - 1$ ，每个顶点和每条边均只被访问常数次数。因此，算法的总运行时间为 $O(n)$ 。
- **空间复杂度：** 需要存储树的结构（邻接表或父亲数组）、每个顶点的状态值 $f(u)$ 以及标记数组。这些数据结构的规模均与顶点数 n 成线性关系。因此，空间复杂度为 $O(n)$ 。

三、结论

对于有 n 个顶点的带权树，通过上述自底向上的贪心算法，可以在 $O(n)$ 的时间复杂度内求得使 T/S 成为 d -森林的最小顶点集 S 。

代码：

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
```

```

4   const int MAXN = 100005;
5   struct Edge {
6       int to, weight;
7   };
8
9   vector<Edge> adj[MAXN];
10  int d, ans = 0;
11  bool possible = true;
12
13  int dfs(int u, int p) {
14      int max_len = 0;
15      bool remove_u = false;
16
17      for (auto& edge : adj[u]) {
18          int v = edge.to;
19          int w = edge.weight;
20          if (v == p) continue;
21
22          if (w > d) {
23              possible = false;
24              return 0;
25          }
26
27          int child_len = dfs(v, u);
28
29          if (child_len + w > d) {
30              remove_u = true;
31          } else {
32              max_len = max(max_len, child_len + w);
33          }
34      }
35
36      if (remove_u) {
37          ans++;
38          return 0;
39      }
40      return max_len;
41  }
42
43  int main() {
44      ios_base::sync_with_stdio(false);
45      cin.tie(NULL);
46
47      int n;

```

```

48     if (!(cin >> n)) return 0;
49
50     for (int i = 1; i ≤ n; i++) {
51         int k; cin >> k;
52         for (int j = 0; j < k; j++) {
53             int v, w;
54             cin >> v >> w;
55             adj[i].push_back({v, w});
56         }
57     }
58     cin >> d;
59
60     dfs(1, 0);
61
62     if (!possible) cout << "No Solution!\n";
63     else cout << ans << "\n";
64
65     return 0;
66 }

```

4-9

4-9 虚拟汽车加油问题。

问题描述：一辆虚拟汽车加满油后可行驶 n km。旅途中有若干加油站。设计一个有效算法，指出应在哪些加油站停靠加油，使沿途加油次数最少。并证明算法能产生一个最优解。

算法设计：对于给定的 n 和 k 个加油站位置，计算最少加油次数。

数据输入：由文件 input.txt 给出输入数据。第 1 行有 2 个正整数 n 和 k ，表示汽车加满油后可行驶 n km，且旅途中有 k 个加油站。接下来的 1 行中有 $k+1$ 个整数，表示第 k 个加油站与第 $k-1$ 个加油站之间的距离。第 0 个加油站表示出发地，汽车已加满油。第 $k+1$ 个加油站表示目的地。

结果输出：将计算的最少加油次数输出到文件 output.txt。如果无法到达目的地，则输出“No Solution”。

输入文件示例

input.txt

7 7

1 2 3 4 5 1 6 6

输出文件示例

output.txt

4

第4-9题答案:

一、算法设计思想

虚拟汽车加油问题的目标是使沿途加油次数最少，这可以通过**贪心算法**来实现。核心策略是：**尽可能行驶到最远的加油站后再进行加油**。即只有当当前油量不足以支撑汽车到达下一个加油站时，才在当前所在的加油站加满油。

具体设计步骤如下：

- 1. 可行性预检查：**遍历输入中相邻两站之间的距离（包括起点到第一站、两站之间、最后一站到目的地，共 $k + 1$ 段距离）。若存在任何一段距离 $d_i > n$ （即单段距离超过了汽车满油后的最大行驶里程），则汽车无法通过该路段。此时应输出 “No Solution!” 并终止计算。
- 2. 状态维护：**设置变量 `sum` 记录加油次数（初始为 0），变量 `s` 记录从上一次加油点（或起点）起行驶的累计距离（初始为 0）。
- 3. 贪心扫描：**顺序遍历每一段路程距离 d_i ：
 - 将当前路段距离累加到 `s` 中，即 `s = s + d_i`。
 - 判断累计距离是否超过最大行驶里程：若 `s > n`，说明在不加油的情况下无法通过当前路段 d_i 。由于已通过预检查确认 $d_i \leq n$ ，因此必须在进入当前路段前的那个加油站加满油。
 - 执行加油操作：加油次数 `sum` 加 1，并重置累计距离 `s`。重置后的 `s` 应等于当前路段的距离 d_i （表示从当前站加满油后开始计算行驶距离）。
- 4. 返回结果：**遍历完所有路段后，`sum` 即为完成旅途所需的最少加油次数。

二、算法正确性证明

1. 贪心选择性质：

设 $G = \{g_1, g_2, \dots, g_m\}$ 是贪心算法选择的加油站序列， $O = \{o_1, o_2, \dots, o_p\}$ 是任意一个最优加油方案。

对于第一次加油，贪心算法选择的是满足 $\sum_{i=1}^{g_1} d_i \leq n$ 的最大 g_1 。显然， o_1 的位置不可能比 g_1 更远（即 $o_1 \leq g_1$ ）。如果 $o_1 < g_1$ ，我们可以将 o_1 替换为 g_1 。由于 g_1 更靠近目的地且在 g_1 处同样是加满油，之后的行驶里程只会比在 o_1 加油时更宽裕，因此替换后的方案依然可行且加油次数不减。重复此过程，可以将最优解 O 逐步转化为贪心解 G ，且保证加油次数 $|G| \leq |O|$ 。由于 O 已是最优，必有 $|G| = |O|$ 。

2. 最优子结构性质：

在做出第一次贪心选择（在 g_1 站加油）后，剩余的问题是：从 g_1 出发，以满油状态完成剩余旅途所需的最少加油次数。这与原问题的结构完全一致，原问题的最优解包含了子问题的最优解。

三、算法分析

- 1. 时间复杂度：**算法主要包含一次对 $k + 1$ 个路段距离的线性扫描。在扫描过程中，每个元素的操作时间为常数级。因此，总时间复杂度为 $O(k)$ ，其中 k 为加油站的数量。
- 2. 空间复杂度：**算法仅使用了常数个辅助变量（如 `sum`，`s`）来记录状态，不随输入规模 k 增加额外空间消耗（不计存储输入的数组）。因此，空间复杂度为 $O(1)$ 。

代码:

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main() {
5      ios_base::sync_with_stdio(false);
6      cin.tie(NULL);
7
8      int n, k;
9      if (!(cin >> n >> k)) return 0;
10
11     vector<int> dist(k + 1);
12     for (int i = 0; i ≤ k; i++) {
13         cin >> dist[i];
14     }
15
16     int ans = 0, s = 0;
17     for (int i = 0; i ≤ k; i++) {
18         if (dist[i] > n) {
19             cout << "No Solution\n";
20             return 0;
21         }
22
23         s += dist[i];
24         if (s > n) {
25             ans++;
26             s = dist[i];
27         }
28     }
29
30     cout << ans << "\n";
31     return 0;
32 }
```

4-11 删数问题。

问题描述：给定 n 位正整数 a ，去掉其中任意 $k \leq n$ 个数字后，剩下的数字按原次序排列组成一个新的正整数。对于给定的 n 位正整数 a 和正整数 k ，设计一个算法找出剩下数字组成的新数最小的删数方案。

算法设计：对于给定的正整数 a ，计算删去 k 个数字后得到的最小数。

数据输入：由文件 input.txt 提供输入数据。文件的第 1 行是 1 个正整数 a 。第 2 行是正整数 k 。

结果输出：将计算的最小数输出到文件 output.txt。

输入文件示例	输出文件示例
input.txt	output.txt
178543	13
4	

第4-11题答案:

一、算法设计思想

删数问题的目标是从一个 n 位正整数中删去 k 个数字，使得剩余数字按原次序排列组成的新数最小。为了使数值最小，应优先保证高位数字尽可能小。本问题采用**贪心算法**，核心策略是：**寻找并删除序列中第一个出现的“峰值”数字**。

具体设计思想如下：

1. 贪心策略（最近下降点优先）：

在整数的位数给定的情况下，判断两个数大小的关键在于从高位到低位出现的第一个不同数字。若要在删除一个数字后使结果最小，应当找到第一个满足 $a[i] > a[i + 1]$ 的下标 i 。此时， $a[i]$ 处于递增序列的末尾，被称为一个“峰值”或“下降点”。删除 $a[i]$ 后，原本在其后的较小数字 $a[i + 1]$ 会占据更高的位权，从而使整体数值达到最小。

2. 算法执行步骤：

- **循环删数：**执行 k 次单次删数操作。
- **单次寻找：**从左向右遍历当前数字序列，比较相邻数字 $a[i]$ 与 $a[i + 1]$ 。
- **执行删除：**
 - * 若发现 $a[i] > a[i + 1]$ ，则删除数字 $a[i]$ ，当前删数操作结束。
 - 若遍历至序列末尾仍未发现下降趋势（即序列为非递减序列，如 12345），则直接删除序列的最后一个数字。
 - * **特殊情况处理：**
 - **前导零：**删数过程中及结束后，若最高位出现了 0（且该数不为单位的 0），应将其滤除，以符合正整数的表示习惯。
 - **空集处理：**若所有数字都被删去，则结果应视为 0。

二、算法分析

1. 正确性分析：

采用数学归纳法证明贪心选择性质。设数字序列为 $S = d_1 d_2 \dots d_n$ 。

若 $d_1 > d_2$ ，删除 d_1 得到的数以 d_2 开头，必然小于删除其他任何位置数字后得到的数（因为其他情况仍以 d_1 开头）。

若 $d_1 \leq d_2 \leq \dots \leq d_i$ 且 $d_i > d_{i+1}$ ，说明前 i 位是单调不减的。此时删除 $d_1 \dots d_{i-1}$ 中任何一个，都会使原本较小的数字被之后较大的数字顶替，而删除 d_i 能够让后面较小的 d_{i+1} 提前进入高位。

由于每一步贪心操作都保证了在当前步下产生了最小的子序列，根据最优子结构性质，连续执行 k 次局部最优选择最终将导向全局最优解。

2. 计算复杂度分析：

- **时间复杂度：**在最坏情况下（如数字序列为单调不减序列），每次删数都需要遍历到末尾。共进行 k 次删数，每次遍历 $O(n)$ ，故总复杂度为 $O(nk)$ 。

注：若引入单调栈数据结构进行优化，可将时间复杂度降低至 $O(n)$ 。

- **空间复杂度：**算法需要存储输入的 n 位数字，空间占用与 n 呈线性关系。因此，空间复杂度为 $O(n)$ 。

代码：

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main() {
5      ios_base::sync_with_stdio(false);
6      cin.tie(NULL);
7
8      string a;
9      int k;
10     if (!(cin >> a >> k)) return 0;
11
12     string res = "";
13     for (char c : a) {
14         while (k > 0 && !res.empty() && res.back() > c) {
15             res.pop_back();
16             k--;
17         }
18         res.push_back(c);
19     }
20 }
```

```

21     while (k > 0 && !res.empty()) {
22         res.pop_back();
23         k--;
24     }
25
26     int start = 0;
27     while (start < (int)res.length() - 1 && res[start] == '0') {
28         start++;
29     }
30
31     if (start == res.length() || res.empty()) cout << "0\n";
32     else cout << res.substr(start) << "\n";
33
34     return 0;
35 }

```

4-15

4-15 最优分解问题。

问题描述：设 n 是一个正整数。现在要求将 n 分解为若干互不相同的自然数的和，且使这些自然数的乘积最大。

算法设计：对于给定的正整数 n ，计算最优分解方案。

数据输入：由文件 input.txt 提供输入数据。文件的第 1 行是正整数 n 。

结果输出：将计算的最大乘积输出到文件 output.txt。

输入文件示例

input.txt

10

输出文件示例

output.txt

30

第4-15题答案:

一、算法设计思想

最优分解问题要求将一个正整数 n 分解为若干互不相同的自然数之和，并使这些自然数的乘积达到最大。根据数学原理，若若干正数之和固定，当这些数越接近时，其乘积越大。为了获得尽可能多的项并保持项之间的差异最小，应采用如下贪心策略：

- 1. 排除无效因子：**在分解中不应包含 1，因为 1 虽占用了和的额度，但对乘积的贡献为 1（即不增加乘积），且会导致因子数减少或数值分布不均。
- 2. 贪心构造序列：**从最小的有效自然数 2 开始，依次将 n 分解为连续的自然数之和（即 2, 3, 4, ...）。这种方式能使因子的个数达到最大，从而有利于乘积的增大。
- 3. 余数处理与均匀分布：**设 $n = 2 + 3 + \cdots + k + r$ ，其中 r 为无法构成下一个连续整数 $k + 1$ 的余

数（即 $r < k + 1$ ）。为了保持因子互不相同且尽可能接近，应将余数 r 分散加到已有的因子中：

- **常规分配：** 若 $r < k$ ，将 r 个单位从序列的末尾（最大因子）开始向前依次加 1。即 $a_k + 1, a_{k-1} + 1, \dots, a_{k-r+1} + 1$ 。
- **特殊分配：** 若余数 r 刚好等于 k （即余数恰好可以分给每一个已有因子），或者在分配过程中会产生重复项，则通过从后向前的顺序递增来确保因子的差异性和互异性。在具体实现中，如果最后剩余的数等于当前最大的因子 $a[k]$ ，为了避免重复，可以将该数再次拆分或通过多轮分配使序列依然保持互异（例如先让 $a[k]$ 增加 1，再分配剩余部分）。

二、算法分析

1. 正确性分析：

- **因子互异性：** 题目明确要求因子互不相同。通过连续自然数构造并在分配余数时采用“从大到小”的原则，可以保证每个因子在原有基础上增加的量是均匀的，从而维持各因子的互异性。
- **贪心选择性质：** 增加因子的个数对乘积的增长比增加单个因子的大小更为显著（例如 $5 + 6 = 11$ 积为 30，而 $2 + 4 + 5 = 11$ 积为 40）。从 2 开始取连续自然数保证了因子的个数 k 最大化。
- **均值不等式原理：** 在和固定的情况下，因子越接近，乘积越大。分配余数时将 r 均匀地补在已有的因子的末尾，使序列仍然保持紧凑分布，最小化了因子间的差值，从而保证了乘积的最优性。

2. 时间复杂度分析：

- **构造阶段：** 将 n 分解为连续自然数 $2 + 3 + \dots + k$ ，已知该和约为 $k^2/2$ ，故 $k \approx \sqrt{2n}$ 。该过程需要执行 $O(\sqrt{n})$ 次加法。
- **分配阶段：** 将余数 r 分配到 k 个因子中。由于 $r \leq k$ ，此过程只需一次线性遍历，时间复杂度为 $O(k) = O(\sqrt{n})$ 。
- **总复杂度：** 算法的总时间复杂度为 $O(\sqrt{n})$ 。

3. 空间复杂度分析：

- 算法需要一个数组存储这 k 个因子。因子的个数 k 与 $\sqrt{n}$ 成正比。因此，空间复杂度为 $O(\sqrt{n})$ 。

代码：

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  struct BigInt {
5      vector<int> digits;
6
7      BigInt(int v) {
8          if (v == 0) digits.push_back(0);
```

```
9         while (v > 0) {
10             digits.push_back(v % 10);
11             v /= 10;
12         }
13     }
14
15     void multiply(int v) {
16         int carry = 0;
17         for (size_t i = 0; i < digits.size(); i++) {
18             long long cur = 1LL * digits[i] * v + carry;
19             digits[i] = cur % 10;
20             carry = cur / 10;
21         }
22         while (carry > 0) {
23             digits.push_back(carry % 10);
24             carry /= 10;
25         }
26     }
27
28     void print() {
29         for (int i = (int)digits.size() - 1; i ≥ 0; i--) {
30             cout << digits[i];
31         }
32         cout << "\n";
33     }
34 };
35
36 int main() {
37     ios_base::sync_with_stdio(false);
38     cin.tie(NULL);
39
40     int n;
41     if (!(cin >> n)) return 0;
42
43     if (n ≤ 4) {
44         cout << n << "\n";
45         return 0;
46     }
47
48     vector<int> a;
49     int sum = 0, cur = 2;
50
51     while (sum + cur ≤ n) {
52         a.push_back(cur);
```

```
53         sum += cur;
54         cur++;
55     }
56
57     int rem = n - sum;
58     int m = a.size();
59
60     if (rem == m) {
61         for (int i = 0; i < m; i++) a[i]++;
62         a[m - 1]++;
63     } else {
64         for (int i = m - 1; i ≥ m - rem; i--) {
65             a[i]++;
66         }
67     }
68
69     BigInt ans(1);
70     for (int x : a) {
71         ans.multiply(x);
72     }
73     ans.print();
74
75     return 0;
76 }
```